ADR记录

ADR-001：采用单体分层架构

状态

已接受

背景

本项目为校园互助平台，功能包括用户管理、需求发布、订单流程、评价体系等，整体业务复杂度中等。项目开发周期较短，团队规模有限，需要在保证系统结构清晰的前提下控制实现复杂度。 同时，系统未来可能存在扩展需求，但当前阶段不具备微服务架构的实施条件。

决策

我们决定采用单体分层架构（Layered Monolithic Architecture）作为系统的整体架构风格，将系统划分为接口层、应用层、领域层与基础设施层。

理由

- 分层架构结构清晰，符合主流开发模式，易于理解与实现
- 相比微服务架构，避免了分布式带来的复杂性（如服务治理、分布式事务）
- 有利于快速开发和迭代，适合课程项目周期
- 校园互助平台既有高频交易逻辑也有低频社交逻辑，单体分层架构可以协调两者的开发需求与性能需要

后果

正面影响：

- 系统结构清晰，开发效率高
- 部署简单，调试方便
- 团队协作成本低

负面影响：

- 随着系统规模增长，可能出现模块耦合增加
- 部分模块扩展性受限

需要关注的风险：

必须严格控制模块边界，避免“巨型单体”问题 需要通过良好的代码结构（如领域划分）降低耦合

AI 辅助记录

AI 初稿内容摘要：

基于项目规模与开发周期，推荐采用单体分层架构作为基础，并强调分层职责划分与扩展性设计。

人工修订内容：

补充说明单体分层架构在该校园平台背景下的性能优势

修订理由：

强调性能是因为校园平台既有高频交易逻辑也有低频社交逻辑，单体分层架构可以更好地协调两者的开发需求与性能需要。 而且对比其他架构，比如微服务架构和事件驱动架构，单体分层架构在性能方面更有优势，尤其是在初期阶段。

ADR-002：引入事件驱动机制实现模块解耦

状态

已接受

背景

系统中存在多个跨模块协作场景，例如订单完成后需要触发信用更新、消息通知以及评价开放等操作。如果采用直接调用方式，会导致模块间强耦合，增加系统复杂度并降低扩展性。

决策

我们决定在系统中引入事件驱动机制（Event-Driven Architecture），在关键业务节点（如订单完成）发布领域事件，并由其他模块通过监听事件进行处理。 同时需要明确事件驱动机制并非作为项目的核心架构，而是作为一种解耦手段，用于特定模块的交互场景，只通过轻量级实现（如 Spring Event）来满足当前需求，避免过度设计。

理由

- 降低模块间直接依赖，实现松耦合设计
- 提高系统扩展性，新增功能只需监听事件即可
- 将核心流程与扩展逻辑解耦，保持主流程简洁
- 可从轻量级实现（Spring Event）逐步演进到消息队列

后果

正面影响：

- 模块之间解耦明显，系统结构更清晰
- 扩展新功能成本低（如新增通知或统计模块）
- 有利于后续架构演进（如引入MQ）

负面影响：

- 事件流增加系统理解难度
- 调试复杂度相对提高

需要关注的风险：

事件滥用可能导致系统行为不可控 需要明确哪些场景使用事件（仅用于非核心流程）
注意事件一致性问题（同步/异步边界）

AI 辅助记录

AI 初稿内容摘要：

建议在跨模块副作用场景中引入事件驱动机制，用于解耦订单、信用和消息模块之间的关系。

人工修订内容：

添加了事件驱动机制的使用范围说明，强调其作为解耦手段而非核心架构，并建议从轻量级实现开始。

修订理由：

强调使用范围是为了避免过度设计，确保事件驱动机制仅用于特定场景，降低系统复杂度，同时也保证了单体分层架构的核心优势。

ADR-003：在领域层引入DDD设计思想

状态

已接受

背景

系统业务涉及订单流转、信用计算、评价体系等核心逻辑，如果全部堆积在Service层，会导致逻辑混乱、难以维护。同时，系统存在多个业务域（订单、内容、信用等），需要进行清晰的业务划分。

决策

我们决定在领域层引入**领域驱动设计（DDD, Domain-Driven Design）**思想，对系统进行领域划分，并在核心业务模块中采用实体（Entity）、仓储（Repository）和领域服务（Domain Service）等模式组织代码。

理由

- 将业务逻辑集中到领域层，避免业务逻辑分散
- 提升代码的可读性与可维护性
- 支持复杂业务规则（如订单状态机）建模
- 与分层架构结合良好，可渐进式引入

后果

正面影响：

- 业务结构清晰，领域边界明确
- 核心逻辑集中，便于维护与扩展

- 有助于团队理解业务模型

负面影响：

- 与单体分层架构的设计融合可能导致部分模块职责不清晰
- 引入一定学习成本（DDD概念较多）
- 代码结构相比传统分层更复杂

需要关注的风险：

- 过度设计（在简单模块中滥用DDD）
- 领域划分不合理可能导致结构混乱
- 需要控制DDD使用范围（优先用于核心域）

AI 辅助记录

AI 初稿内容摘要：

建议在领域层中引入DDD思想，对核心业务模块进行建模，并结合分层架构实现高内聚设计。

人工修订内容：

补充说明将轻量化DDD思想引入将与单体分层架构矛盾的风险，并强调需要控制DDD使用范围，优先用于核心域，避免过度设计。

修订理由：

强调风险是因为DDD设计思想虽然有助于提升代码质量，但如果滥用或与单体分层架构设计不当结合，可能导致系统结构混乱，增加开发复杂度。 因此需要明确DDD的使用范围和原则，确保其作为提升代码质量的工具而非增加系统复杂度的负担。